

Gato (Tic-Tac-Toe) con Negamax

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Lógica del Juego	2
2.1. Representación del estado	2
2.2. Niveles de dificultad	2
3. Algoritmo Negamax	2
3.1. Fundamento matemático	2
3.2. Pseudocódigo	2
3.3. Factor de profundidad	3
4. Arquitectura del Código	3
5. Flujo de Ejecución	3
6. Comparativa Minimax vs Negamax	3

1. Descripción del Proyecto

Implementación del juego **Gato** con inteligencia artificial basada en **Negamax**, una variante simplificada del algoritmo Minimax. A diferencia de la versión Minimax, este proyecto incluye **tres niveles de dificultad**: Fácil, Medio y Difícil.

2. Lógica del Juego

2.1. Representación del estado

El tablero de 3×3 se almacena en un arreglo unidimensional de 9 enteros. Cada celda puede ser VACÍA, JUGADOR o CPU.

2.2. Niveles de dificultad

- **Fácil**: la CPU elige movimientos *aleatorios* sin ninguna evaluación.
- **Medio**: la CPU aplica Negamax con **profundidad limitada** (2-3 niveles), lo que la hace imperfecta.
- **Difícil**: la CPU aplica Negamax **completo** sin límite de profundidad, siendo imbatible.

3. Algoritmo Negamax

3.1. Fundamento matemático

Negamax se basa en la simetría de los juegos de suma cero:

$$\text{minimax}(\text{MAX}, s) = -\text{minimax}(\text{MIN}, s)$$

Esto permite escribir **una sola función** en lugar de dos (maximizador y minimizador).

3.2. Pseudocódigo

```
1 función negamax(tablero, profundidad, jugadorActual):
2     si hayGanador(tablero, jugadorActual):
3         retornar +10 - profundidad
4     si hayGanador(tablero, oponente(jugadorActual)):
5         retornar -10 + profundidad
6     si tableroLleno(tablero):
7         retornar 0
8
9     mejorPuntuacion = -infinito
10    para cada celda vacía en tablero:
11        tablero[celda] = jugadorActual
12        puntuacion = -negamax(tablero, profundidad+1,
13                               oponente(jugadorActual))
14        tablero[celda] = vacío
15    mejorPuntuacion = max(mejorPuntuacion, puntuacion)
```

16
17

```
retornar mejorPuntuacion
```

El signo negativo antes de la llamada recursiva invierte la perspectiva: lo que es bueno para el oponente es malo para mí.

3.3. Factor de profundidad

La puntuación ajustada por profundidad ($+10 - \text{profundidad}$) hace que el algoritmo prefiera victorias rápidas y derrote lo más tarde posible.

4. Arquitectura del Código

El proyecto se divide en dos clases:

- `Clase_Gato.java`: contiene la lógica del juego, el algoritmo Negamax y las utilidades de evaluación del tablero.
- `Gato_Grafico.java`: contiene la interfaz gráfica Swing, el menú de selección de dificultad y los listeners de eventos.

5. Flujo de Ejecución

1. Al iniciar, se muestra un diálogo para seleccionar el nivel de dificultad.
2. El jugador humano hace clic en una celda vacía.
3. Se registra el movimiento y se verifica estado terminal.
4. Dependiendo del nivel, la CPU:
 - *Fácil*: elige celda aleatoria.
 - *Medio*: ejecuta Negamax con profundidad máxima 3.
 - *Difícil*: ejecuta Negamax completo.
5. Se actualiza la GUI y se verifica nuevamente el estado terminal.

6. Comparativa Minimax vs Negamax

Característica	Minimax	Negamax
Número de funciones	2 (max + min)	1 (unificada)
Complejidad del código	Mayor	Menor
Resultado	Idéntico	Idéntico
Dificultades variables	Más complejo	Más sencillo